

Evaluating Source Code Embeddings from LLMs for Educational Downstream Tasks

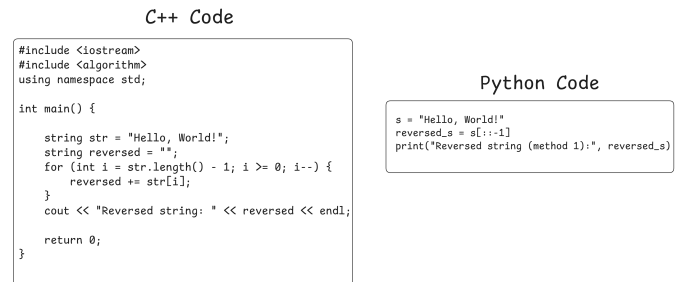
Abstract—Recent advancements in deep learning (DL) have significantly impacted natural language processing tasks such as language modeling, translation, and paragraph comprehension. These developments extend naturally to the software engineering domain, particularly in learning from source code. This work focuses on optimizing DL applications in source code learning by analyzing how different input embeddings affect model performance when training on C++ code. Given that source code differs from natural language in structure and semantics, we introduce a framework to systematically compare multiple embedding strategies for three educational downstream tasks: code correctness, code summarization, and introduce a framework for code similarity. We also propose a fine-tuning approach for adapting a pre-trained, code-centric large language model to generate context-aware C++ embeddings tailored to these tasks and explore their performance on these downstream tasks. Experimental evaluations reveal key findings on features associated with C++ source code and their respective embeddings at varying levels of problem complexity. We also discuss limitations and drawbacks of code embeddings generated by such models in classifying correct versus incorrect code, and how these findings can support automated and accurate grading in educational settings. In addition, we evaluate the models’ source code summarization capabilities, highlighting how well they generate meaningful summaries that capture both structural and semantic aspects of the code.

Index Terms—educational data mining, machine learning, deep learning, Code-Centric Large Language Models (LLMs).

I. INTRODUCTION

Deep Learning (DL) has recently become a significant area within Machine Learning (ML) due to its remarkable success in fields like Computer Vision and Natural Language Processing (NLP). In NLP, DL models have been proven to significantly enhance the performance of traditional tasks such as semantic role labeling, named entity recognition [19], machine translation [24], and question answering [19]. Similarly, source code, a specialized form of structured language created by programmers, can also be effectively analyzed by DL models. The capacity of artificial intelligence to analyze and produce complex software constructs, such as task-specific source code in diverse programming languages, plays a crucial role in Software Engineering. Recent advances focus on the use of machine learning and deep learning techniques to effectively model source code [11]. Deep learning techniques for source code modeling focus on transforming source code into a format that can be interpreted by machines. In the educational domain, source code modeling can have multiple applications, one of which is code summarization. Code sum-

marization refers to the process of creating a natural language description for a given piece of source code or function in a programming language. This summary generally explains the various methods and classes used in the code and provides a clear outline of the intended functionality of the code. The task of code summarization varies between each language due to the different nuances and the general syntax associated with each language. Figure 1 depicts two code snippets performing the same task but in different programming languages (C++ and Python) to illustrate the structural differences between the two languages. Given that source code differs from natural language in structure and semantics, we introduce a framework to systematically compare multiple embedding strategies for three educational downstream tasks: code correctness, code summarization, and introduce a framework for code similarity.



C++ Code

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    string str = "Hello, World!";
    string reversed = "";
    for (int i = str.length() - 1; i >= 0; i--) {
        reversed += str[i];
    }
    cout << "Reversed string: " << reversed << endl;
    return 0;
}
```

Python Code

```
s = "Hello, World!"
reversed_s = s[::-1]
print("Reversed string (method 1):", reversed_s)
```

Fig. 1. C++ vs Python code snippets

II. RELATED WORK

The rapid advancements in deep learning for software engineering have given rise to various distributed code representation techniques, known as embeddings. Each method employs a distinct algorithm to encapsulate contextual information while preserving generalizability. A token in NLP refers to an individual unit extracted from a larger body of text, which can be a word, subword, or character, depending on the granularity of the tokenization process. Tokenization is one of the first steps in preparing text for machine learning models and is crucial for analyzing and understanding language [10], [22], [23]. The following section discusses key embedding approaches that are extensively used in code summarization and generalization tasks.

A. Word2vec

Word2Vec [2], introduced by Mikolov et al. (2013), is a neural network-based approach for learning distributed representations of words based on their contextual co-occurrences. Originally designed for natural language processing, it has been adapted for source code modeling by treating code tokens as words and sequences of code as sentences. This adaptation enables Word2Vec to capture semantic and syntactic relationships between code elements, aiding tasks such as code summarization, clone detection, and vulnerability identification. The model is particularly effective for applications like semantic code search, bug detection, and code completion due to its ability to learn meaningful token embeddings. However, Word2Vec inherently disregards structural aspects of code, such as abstract syntax trees (ASTs) and control flow graphs (CFGs), as it processes code in a linear fashion. Additionally, it generates static embeddings, limiting its ability to dynamically adapt to different programming contexts.

B. Glove

GloVe (Global Vectors for Word Representation) [18], introduced by Pennington et al. (2014), is a word embedding technique that constructs vector representations based on word co-occurrence statistics from a given corpus. Unlike Word2Vec, which relies on local context windows, GloVe captures global word relationships by factorizing a word co-occurrence matrix, enabling it to encode both syntactic and semantic relationships effectively. When adapted for source code modeling, GloVe learns token representations by analyzing their co-occurrence patterns across large codebases, aiding tasks such as code summarization, clone detection, and vulnerability identification. Its reliance on matrix factorization allows it to model long-range dependencies in code, making it particularly useful for understanding functionally similar code components. However, like Word2Vec, GloVe produces static embeddings that do not dynamically adapt to varying code contexts. Additionally, it does not inherently capture structural relationships, such as those present in abstract syntax trees (ASTs) or control flow graphs (CFGs).

C. Transformer model based Embeddings

Contextual embeddings, typically derived from transformer-based models [24] such as CodeBERT [6] and CodeT5 [25], adjust token representations based on their context. These models process the entire input sequence to compute embeddings that capture the contextual semantics of each token. A transformer-based embedding model leverages the transformer architecture to generate vector representations, or embeddings, of input data while capturing contextual dependencies. Introduced in "Attention is All You Need" (Vaswani et al., 2017) [24], the transformer employs a self-attention mechanism that evaluates the relevance of different parts of the input sequence, regardless of their position. In natural language processing (NLP), transformer-based models such as BERT, GPT, and T5

process text in parallel and refine token embeddings through multiple layers of self-attention and feed-forward networks. These embeddings encode semantic meaning and inter-token relationships, serving as versatile representations for various tasks, including text classification and machine translation. Pre-training a model involves training it on a large corpus of data to learn general representations, where it captures patterns and contextual relationships through unsupervised or self-supervised techniques. This pretraining phase improves the model's ability to generalize, allowing it to perform well across diverse downstream tasks with minimal task-specific data. Transformer-based models have shown superior performance in many NLP tasks because of their ability to capture long-range dependencies and efficiently process input sequences.

- **CodeBERT**: A transformer-based model pre-trained on a large corpus of source code. CodeBERT is a bimodal pre-trained model for programming language (PL) and natural language (NL) tasks. It is designed to learn general-purpose representations that support downstream NL-PL applications such as natural language code search and code documentation generation. CodeBERT is developed using a Transformer-based neural architecture and is trained with a hybrid objective function that incorporates the pre-training task of replaced token detection (RTD). This approach enables the utilization of both bimodal data of NL-PL pairs and unimodal data, where the former provides input tokens for model training while the latter helps to learn better generators [5]. CodeBERT uses a multi-layer bidirectional Transformer as its model architecture and employs two training objectives: Masked Language Modeling (MLM) and RTD [6]. The model contains approximately 125 million parameters and has demonstrated state-of-the-art performance on various NL-PL tasks, showcasing its ability to capture both syntactic and semantic information in code [6], [12].
- **CodeT5**: Another transformer-based model specifically designed for code understanding and generation tasks. In order to capture more code-specific features, we propose to leverage token type information from code. CodeT5 focuses on the type of identifiers (e.g., function names and variables) as they are one of the most PL-agnostic features and reserve rich codesemantics [25]. Specifically, we convert the PL segment into an Abstract Syntax Tree (AST) and extract the node types for each code token. Finally, it constructs a sequence of binary labels $y \in \{0, 1\}^m$ for the PL segment, where each $y_i \in \{0, 1\}$ represents whether the code token c_i is an identifier or not. [5]

In this study, we leverage context-aware embeddings generated by transformer-based models to evaluate their effectiveness on the specified downstream educational tasks within our experimental pipelines.

D. Code embeddings and correctness

Recent advancements in code representation learning have explored approaches that move beyond syntactic similarity to capture functional and semantic properties of source code. CodeT [1] introduces an execution-based strategy for improving code generation quality by generating candidate source code snippets alongside corresponding test cases and validating them through behavioral consistency checks. This approach leverages source code execution as an implicit functional signal, enabling more reliable selection of correct solutions. In contrast, Contrastive Code Representation Learning (ContraCode) [9] employs a semantic-aware contrastive learning framework that generates functionally equivalent source code variants using source-to-source transformations. By optimizing embeddings to be invariant to such transformations, ContraCode learns representations that emphasize source code functionality rather than surface-level syntax. Both methods significantly advance the richness of code embeddings: CodeT, through dynamic validation and execution-driven signals, and ContraCode, through semantic invariance in representation learning, improving performance across tasks such as code summarization, type inference, and clone detection.

E. Datasets

- **Project CodeNet** [20] is a large-scale dataset introduced to advance AI-for-code research. It contains approximately 14 million code samples across 55 programming languages, with a total of about 500 million lines of code collected from online judge systems such as Aizu Online Judge and AtCoder. The dataset includes rich metadata, such as problem descriptions, sample input/output pairs, execution status, runtime, and memory usage, enabling both supervised and evaluation tasks. CodeNet provides benchmarks for a wide range of tasks, including code similarity detection, code classification, source code translation, and performance prediction. In addition, it offers tools for data preprocessing and model-ready representation generation, making it a comprehensive resource for developing and evaluating models for programming-related tasks.
- The **Google Code Jam (GCJ)** [14] dataset originates from submissions to the Google Code Jam competitive programming contest. The GCJ-297 subset consists of approximately 297 problems and around 280,000 code submissions in multiple programming languages, primarily C++, Java, Python, and C. The dataset captures a variety of coding patterns and problem-solving strategies across different participants and languages, making it useful for tasks such as code classification and code similarity detection.

In our experiments, the GCJ dataset is employed to assess the correctness of code snippets, while the CodeNet dataset is used to evaluate the effectiveness of embeddings for code summarization tasks.

III. METHODOLOGY

A. Experimental Setup

- **SCVP**(Source Code Verification Pipeline): As introduced in our earlier work [16], the SCVP experimental framework (refer section III-B) is formulated to provide a rigorous evaluation of code embedding methods within the context of a binary classification task. We utilize a dataset comprising 3,000 C++ code samples from the GCJ dataset, curated from a larger corpus of 5,000 samples originating from the year 2009. To address class imbalance issues, we employ undersampling techniques, ensuring a balanced representation of classes in our final dataset [8]. Addressing class imbalance is crucial in binary classification tasks, as it prevents the model from developing a bias towards the majority class, which can undermine its ability to accurately classify the minority class. In the absence of such measures, the model may exhibit a skewed performance, predominantly predicting the majority class and yielding misleadingly high accuracy scores while failing to properly identify the minority class. To mitigate this issue, we employ undersampling techniques, reducing the representation of the majority class to ensure a balanced dataset, thereby enhancing the model's generalization and classification capability across both classes.
- **SCVP(fine-tuned)**: This pipeline (refer section III-B1) was developed to assess the functional correctness of code submissions using a fine-tuned CodeBERT model for binary classification and is the extension of SCVP. The training dataset was sourced from the Google Code Jam (GCJ) archives and comprised a balanced mix of 100,000 correct and 100,000 incorrect submissions. These labels were derived from the problem metadata associated with the contests. The goal of the pipeline was to classify whether a given code snippet correctly solves its associated problem, without executing the code. Fine-tuning experiments were conducted across various settings, including different numbers of training epochs, learning rates, and label configurations, to explore their effects on model behavior. The pipeline uses the sequence classification head of CodeBERT to output a confidence score between 0 and 1, indicating the model's belief that a submission is functionally correct. Despite initial issues such as class imbalance and limited variance in model outputs, these were addressed through balanced data sampling and careful evaluation procedures. This approach enables automated filtering and ranking of candidate code solutions based on learned correctness patterns.
- **SCSP**(Source Code Summarization pipeline): This pipeline (refer section III-C) was designed to evaluate the source code summarization capabilities of language models by generating and assessing summaries of code snippets. We utilize the DeepSeek's-R1-Distill-Qwen-

32B model to generate summaries for code snippets drawn from the CodeNet dataset, and evaluate the quality of these summaries using Bleu [17] and BERTScore [28] as primary evaluation metrics. A subset of 20,000 samples from the CodeNet dataset is employed to evaluate the model’s effectiveness in generating precise and informative code summaries. Owing to computational constraints, the generation and evaluation of summaries are performed in batches of 5,000 samples.

We propose three pipelines for testing and evaluating C++ source code embedding in three specific tasks: code summarization, code correctness, and code similarity. The pipelines are described as follows:

- SCVP (Source Code Verification Pipeline)
- SCSP (Source Code Summarization Pipeline)
- SCLP (Source Code Likeness Pipeline)

We leverage the above three pipelines in order to test the fine-tuned model’s embeddings.

B. SCVP(Source Code Verification Pipeline)

We leverage the SCVP to evaluate the performance of code embeddings in the downstream task of classifying code as correct or incorrect. First, the C++ code snippets are converted into code embeddings using CodeT5 and CodeBERT. Representing tokens as numerical values is a crucial step in text mining to enable the use of machine learning and deep learning models. This project proposes the use of the contextual embeddings generated by CodeT5 and CodeBERT to transform each C++ source code snippet into numerical form. In this study, the embeddings function as feature representations and are treated synonymously throughout the analysis. The embedding techniques under investigation exhibit distinct methodologies for encoding semantic information and representing context-dependent token meanings within their respective embeddings. CodeT5 and CodeBERT exhibit fundamental differences in their architectures and training approaches. CodeT5, structured as an encoder-decoder model, facilitates both code generation and interpretation, benefiting from a diverse set of pre-training objectives designed specifically for source code tasks. A key aspect of CodeT5’s pre-training is span prediction, where contiguous spans of tokens are masked, and the model is trained to reconstruct them, improving its ability to generate coherent and contextually accurate code snippets. Additionally, CodeT5 incorporates identifier-aware pre-training, a critical feature for improving the model’s understanding of programming constructs like variable names, functions, and other identifiers. In this pre-training step, CodeT5 learns to predict the relationship and usage of identifiers within a specific code context. This allows the model to capture important patterns, such as how function names or variable names are used and interact with each other in different coding scenarios, improving the model’s ability to generate meaningful code completions, variable suggestions, and error detections. This identifier-awareness is particularly beneficial for tasks such as

code summarization, debugging, and automatic code translation, where understanding the role of specific identifiers is crucial.

In contrast, CodeBERT, an encoder-based model adapted from RoBERTa, focuses exclusively on understanding source code and its alignment with natural language. It relies on masked language modeling (MLM), where individual tokens are randomly masked, and the model predicts the missing words, thereby learning contextual dependencies within the code. Additionally, CodeBERT employs replaced token detection (RTD), a contrastive learning approach that distinguishes between correct tokens and plausible yet incorrect substitutions. This mechanism reinforces CodeBERT’s ability to discern meaningful code representations, making it particularly effective for tasks such as code search, classification, and defect detection [13], [25], [27]. Both CodeBERT and CodeT5 models, despite their architectural differences, produce high-dimensional embeddings that capture rich semantic and syntactic features of code tokens [6], [25].

To enhance efficiency in downstream tasks and mitigate computational overhead, we utilize mean pooling as a dimensionality reduction approach. This method processes embeddings, which vary in length depending on the code snippet, by aggregating them along the sequence axis—the dimension representing token positions in the snippet. As a result, we obtain a compact, dense vector representation for each snippet, facilitating further analysis and learning tasks. The dimensionality of this mean-pooled vector corresponds to the hidden size of the respective models, which is 768 for both CodeBERT and CodeT5 [6], [25].

For practical implementation and to ensure compatibility with various machine learning frameworks, we convert these mean-pooled vectors into list structures. This transformation allows for more flexible manipulation and integration into subsequent analysis pipelines while preserving the semantic richness captured by the original embeddings. Figure 2 illustrates a working example of SCVP.

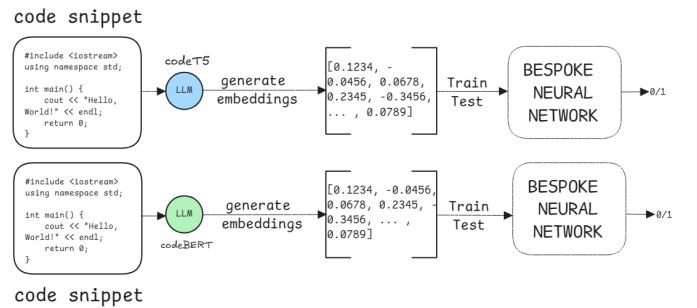


Fig. 2. Source Code Verification Pipeline (SCVP)

We design and implement a custom neural network architecture comprising an input layer with 768 neurons, followed by two hidden layers with 512 and 256 neurons, respectively. The output layer consists of a single neuron with a sigmoid

activation function to facilitate binary classification. The model is trained using binary cross-entropy as the loss function. Figure 3 depicts the neural network used to classify the code snippets.

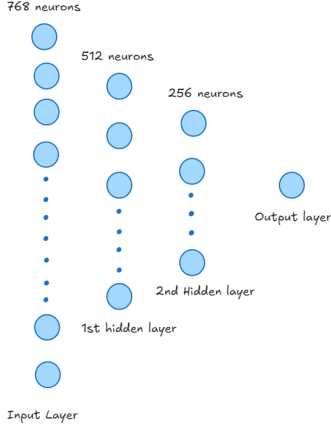


Fig. 3. Bespoke Neural Network

1) *SCVP(Source Code Verification Pipeline - Fine tuned)*: We extend SCVP by leveraging the high computational capacity and long-range attention mechanisms of code language models to assess C++ source code embeddings for the task of correctness classification. As illustrated in Figure 2, the extended SCVP framework involves fine-tuning a code language model on these embeddings to distinguish between correct and incorrect code.

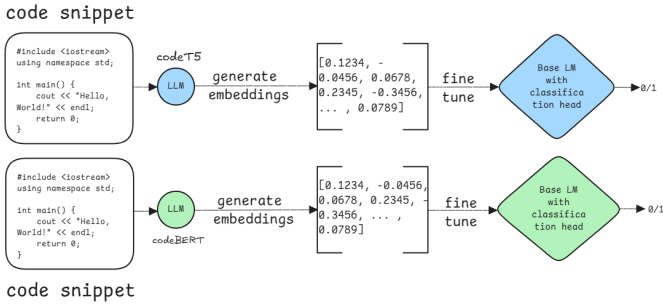


Fig. 4. Example of Fine-tuned SCVP

C. SCSP (Source Code Summarization Pipeline)

We utilize the SCSP framework to assess the summarization capabilities, a working example of SCSP. Through prompt engineering, we guide the model to produce summaries that adhere to a manually curated template, ensuring consistency across generated outputs. Given that both the ground truth (problem description) and the generated summaries are in natural language, we employ standard natural language evaluation metrics to measure the similarity between them. Specifically, our study uses BERTScore [28] and Bleu [17] to evaluate how closely the generated summaries align with the actual problem

descriptions of the code snippets. Figure 5 depicts a working example of SCSP.

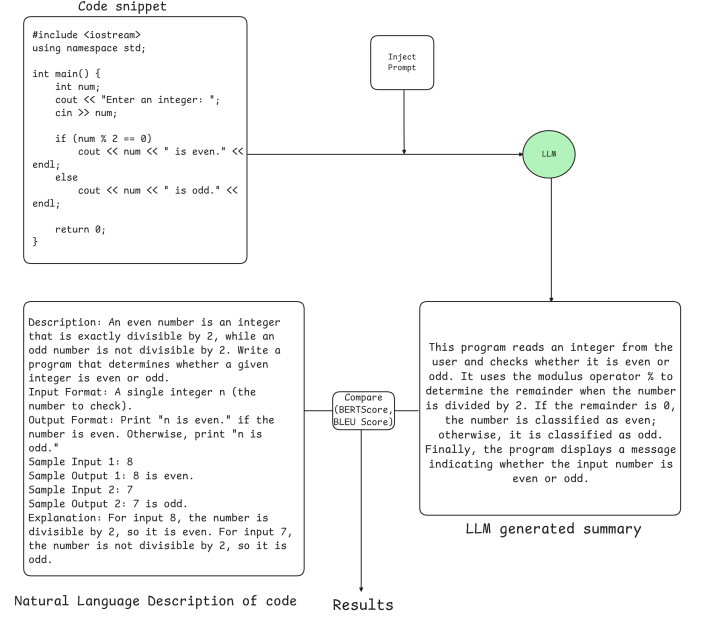


Fig. 5. Example of Source Code Summarization Pipeline (SCSP)

We make use of the deepseek CoT model in order to generate the summaries in the template shown in figure 6.

Problem description	formatted problem description
Write a function that checks whether a given integer is a prime number. The function should take an integer n as input and should return a boolean value, true if the number is prime and false otherwise.	Function Name: isPrime Purpose: To determine whether a given integer is a prime number. Input: An integer n. Output: A boolean value - true if n is prime, false otherwise. Key Logic: Return false for inputs less than or equal to 1, loop from 2 to n and check if n is divisible by any number in that range. If divisible, return false; else, return true. Dependencies: Standard library (for basic input/output if needed). Edge Cases: n = 0 or n = 1 → Not prime, Negative numbers → Not prime and n = 2 → Prime (smallest prime number).

Fig. 6. problem description v/s formatted template

The template encompasses key aspects of the code like the function name, its purpose, inputs/outputs, key logic, dependencies, edge cases, and an example usage.

IV. RESULTS AND KEY FINDINGS

The results of SCVP and its fine-tuned variant are presented in Tables I, II and III. Figure 7 illustrates the comparative performance of embeddings generated by CodeT5 and CodeBERT within the SCVP framework. Findings from both versions indicate that the notion of code correctness remains loosely captured, and these observations are further analyzed in Section V.

The results for the SCSP are depicted in table IV. The results show that the average BLEU scores across all chunks are relatively low, ranging from 0.0175 to 0.0304. This is expected, as BLEU relies on n-gram overlap between the

Metric	CodeBERT	CodeT5
Final Loss (Epoch 10)	0.6452	0.6604
Train Accuracy	0.6328	0.6182
Test Accuracy	0.5867	0.6200
Train Precision	0.6008	0.5788
Test Precision	0.5655	0.6009
Train Recall	0.7888	0.8311
Test Recall	0.7715	0.8291
Train F1 Score	0.6821	0.6824
Test F1 Score	0.6527	0.6968

TABLE I
PERFORMANCE COMPARISON OF CODEBERT AND CODET5 (BEST VALUES IN **BOLD**) FOR SCVP

Metric	CodeBERT	CodeT5
Train Accuracy	0.6157	0.6290
Test Accuracy	0.5745	0.5849
Train Precision	0.5777	0.6092
Test Precision	0.5507	0.5700
Train Recall	0.8777	0.7522
Test Recall	0.8327	0.6951
Train F1 Score	0.6956	0.6653
Test F1 Score	0.6613	0.6197

TABLE II
CROSS-VALIDATION PERFORMANCE COMPARISON OF CODEBERT AND CODET5 (BEST VALUES IN **BOLD**) FOR SCVP

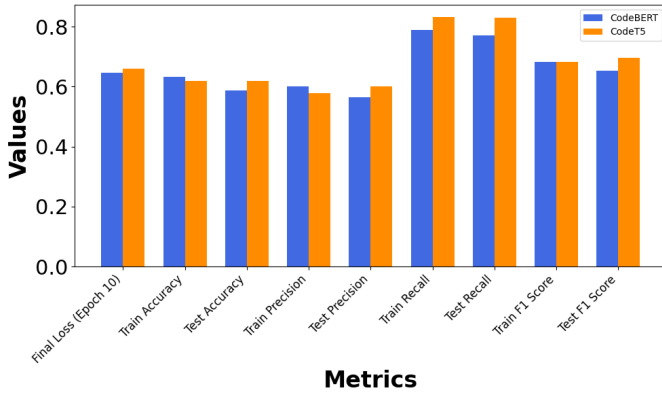


Fig. 7. SCVP Evaluation Metrics (CodeT5 v/s CodeBERT)

Metric	Value
Evaluation Loss	1.2947
Evaluation Accuracy	0.5400
Evaluation F1 Score	0.5461
Evaluation Runtime (s)	88.5169
Samples per Second	6.7780
Steps per Second	0.8470
Epoch	10.0

TABLE III
SCVP(FINE-TUNED): EVALUATION METRICS AT EPOCH 10

Chunk	Average BLEU Score	Average BERTScore F1
1st Chunk	0.0304	0.8154
2nd Chunk	0.0300	0.8151
3rd Chunk	0.0269	0.8136
4th Chunk	0.0175	0.8114
Total Average	0.0262	0.8139

TABLE IV
CHUNK-WISE BLEU AND BERTSCORE RESULTS FOR SCSP.

generated and reference summaries, which may not fully capture semantic similarity or contextual understanding. In contrast, BERTScore achieves consistently high F1 values (approximately 0.8114 to 0.8154), as it leverages contextual embeddings to assess similarity at the semantic level. This is particularly important in code summarization, where the ability to understand the underlying functionality and produce informative summaries matters more than exact token overlap. Furthermore, by manual inspection by computer science experts of the generated summaries, we observe that the model captures the essential behavior and logic of the code effectively, supporting the high BERTScore values despite the low BLEU scores.

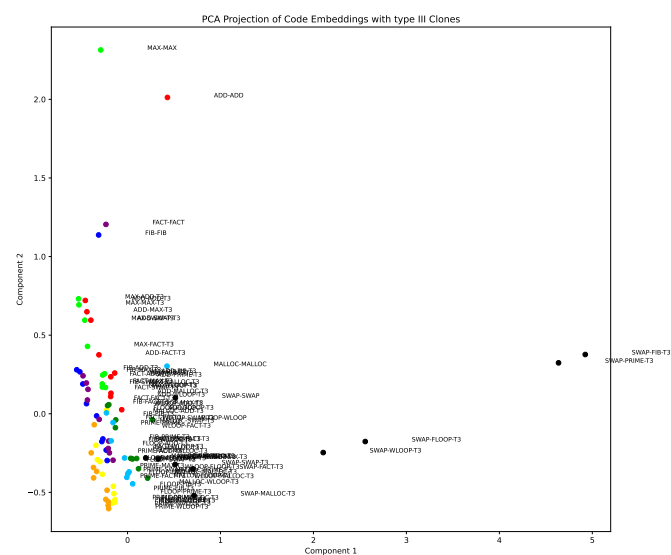
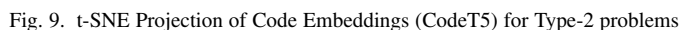
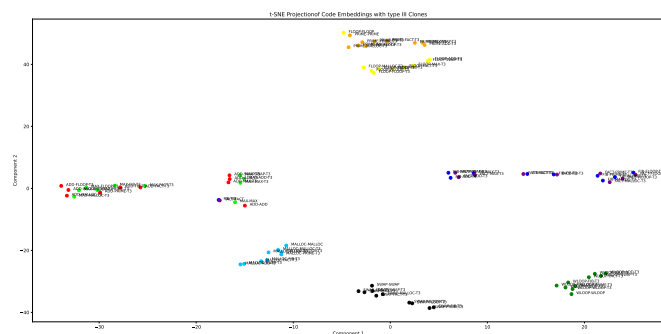
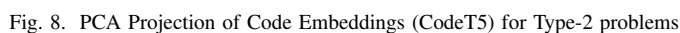
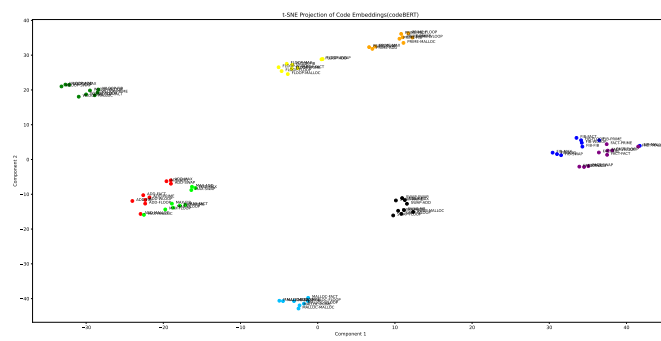
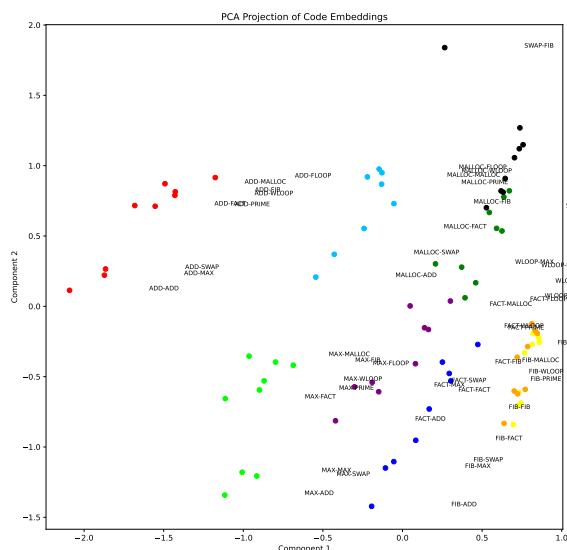
V. DISCUSSION ON SCVP

Although the Source Code Summarization Pipeline (SCSP) demonstrated encouraging performance, the findings from the Source Code Verification Pipeline (SCVP) reveal that the models fail to learn any meaningful signal related to correctness from the generated embeddings, leading to the conclusion that code correctness is not effectively captured within these representations. This observation motivated a deeper investigation into the representation of correctness in code embeddings and how such a property might be more explicitly encoded. As an initial step, we examine whether code snippets of minimal difficulty and complexity are adequately represented by the embeddings, aiming to gain further insight into the specific features these embeddings capture. We adopt four types of software clones at different levels of complexity and difficulty [21]:

- Type-1: Two code fragments are exactly the same, except for comments and layouts.
- Type-2: In addition to Type-1 clone differences, the two code fragments are different in identifier names and literal values.
- Type-3: In addition to Type-1 and Type-2 clone differences, one of the two code fragments was revised, added, or deleted relative to the other one. In other words, they are syntactically similar at the statement level.
- Type-4: Two code fragments implement the same functionality, while they are not similar syntactically.

Type-1 problems were excluded due to their negligible impact on complexity. To analyze embedding behavior, we utilize code clones corresponding to Type-2, Type-3, and Type-4 problems for generating their respective embedding projections. The problems in real-world scenarios like the ones in the GCJ dataset can be categorized into Type-4 problems based on the definition of Type-4 problems in section V.

The t-SNE and PCA visualizations of embeddings generated by CodeT5 and CodeBERT for Type-2, Type-3, and Type-4 problems are presented in figures 11, 10, 8, 9, 12, 13, 15 and 14. In these visualizations, code clone samples of the same problem type are color-coded (e.g., red, blue, green, orange). Type-1 problems were excluded from the analysis due to their



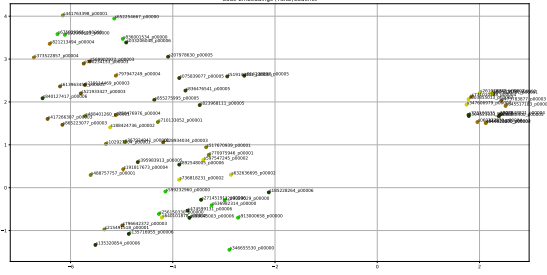


Fig. 14. t-SNE projection of CodeNet (CodeT5)

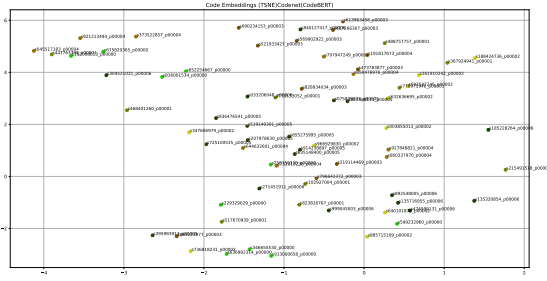


Fig. 15. t-SNE projection of CodeNet (CodeBERT)

low complexity and limited relevance to real-world scenarios. It is important to note that function names are irrelevant in this context; the primary focus of these results lies in the clustering patterns.

Type 2 problems exhibited clear clustering, as shown in Figures 10 and 11 for CodeBERT embeddings, and in Figures 8 and 9 for CodeT5 embeddings. It can be observed that Type-2 problem embeddings organize into eight distinct clusters, with significant separation between each cluster. These results indicate that the learned embeddings successfully capture structural and semantic similarities among Type-2 problem instances.

However, the clustering quality deteriorates for Type-3 problems, as observed in Figures 13 and 12. For Type-4 problems, which are more representative of real-world code samples such as those in the CodeNet dataset, embeddings generated by both CodeT5 and CodeBERT exhibit almost no meaningful clustering, as illustrated in Figures 14 and 15.

These observations suggest that while the models can effectively identify similarities for simpler problem types, they struggle to generalize to complex, real-world code, leading to a lack of coherent grouping in the embedding space. This limitation reflects the models' reduced ability to infer semantic similarity and functional correctness in more challenging scenarios. Due to space limitations, we have included only a subset of the results. Readers interested in additional experimental outcomes and complete datasets are encouraged

to visit our GitHub repository at [15].

VI. CONCLUSION AND FUTURE WORKS

In this work, we evaluated the capabilities of large language models for code understanding, summarization, and similarity analysis. Using SCVP, we assessed code correctness, showing that models perform well on simpler problems but struggle with complex, real-world code. We also analyzed source code summarization with DeepSeek [7] using Bleu [17] and BERTScore [28], revealing limitations in capturing intricate logic. Additionally, we introduce the SCLP (Source Code Likeness Pipeline), which quantifies embedding similarity using cosine similarity, Manhattan distance, and Euclidean distance, providing a comprehensive measure of code likeness. Overall, our results highlight both the potential and current limitations of these models in educational and real-world coding tasks. In future work, we plan to scale our experiments to the entire CodeNet dataset to further validate and extend our findings.

A. SCLP(Source Code Likeness Pipeline)

We introduce the SCLP framework to evaluate the similarity between code embeddings generated from C++ code snippets using two embedding models, CodeBERT and CodeT5. The embeddings derived from the code samples are projected into a vector space, and similarity scores are computed using cosine similarity [26], Manhattan distance [3], and Euclidean distance [4]. Cosine similarity is employed to capture the angular alignment between embeddings, making it effective for measuring directional similarity, irrespective of vector magnitude. Manhattan distance, based on the L1 norm, is used to quantify the absolute differences across dimensions, while Euclidean distance, based on the L2 norm, measures the straight-line distance between embeddings in the vector space. Together, these metrics provide a comprehensive assessment of both directional- and magnitude-based similarities.

REFERENCES

- [1] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022.
- [2] Kenneth Ward Church. Word2vec. *Natural Language Engineering*, 23(1):155–162, 2017.
- [3] Susan Craw. Manhattan distance. In *Encyclopedia of Machine Learning and Data Mining*. Springer, 2017.
- [4] Per-Erik Danielsson. Euclidean distance mapping. *Computer Graphics and image processing*, 14(3):227–248, 1980.
- [5] Zishuo Ding, Heng Li, Weiyi Shang, and Tse-Hsun Peter Chen. Can pre-trained code embeddings improve model performance? revisiting the use of code embeddings in software engineering tasks. *Empirical Software Engineering*, 27(3):63, 2022.
- [6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.
- [7] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

- [8] Haibo He and Edwardo A Garcia. Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering*, 21(9):1263–1284, 2009.
- [9] Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph Gonzalez, and Ion Stoica. Contrastive code representation learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021.
- [10] Daniel Jurafsky. *Speech and language processing*, 2000.
- [11] Triet HM Le, Hao Chen, and Muhammad Ali Babar. Deep learning for source code modeling and generation: Models, applications, and challenges. *ACM Computing Surveys (CSUR)*, 53(3):1–38, 2020.
- [12] Triet Huynh Minh Le, M Ali Babar, and Tung Hoang Thai. Software vulnerability prediction in low-resource languages: An empirical study of codebert and chatgpt. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, pages 679–685, 2024.
- [13] Yue Liu, Chakkrit Tantithamthavorn, Yonghui Liu, and Li Li. On the reliability and explainability of language models for program generation. *ACM Trans. Softw. Eng. Methodol.*, 33(5), June 2024.
- [14] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. Concode: Code understanding and generation from natural language. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2016.
- [15] Neil Allister Pais. Evaluating source code embeddings from llms for educational downstream tasks. <https://github.com/neilpais/Evaluating-Source-Code-Embeddings-from-LLMs-for-Educational-Downstream-Tasks>, 2025.
- [16] Neil Allister Pais and Ritu Chaturvedi. C++ source code verification pipeline.
- [17] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [18] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- [19] Matthew E Peters, Waleed Ammar, Chandra Bhagavatula, and Russell Power. Semi-supervised sequence tagging with bidirectional language models. *arXiv preprint arXiv:1705.00108*, 2017.
- [20] Ruchir Puri, David Kung, Viet Nguyen, et al. Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. In *NeurIPS 2021 Datasets and Benchmarks Track*, 2021.
- [21] Hitesh Sajjani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K Roy, and Cristina V Lopes. Sourcerercc: Scaling code clone detection to big-code. In *Proceedings of the 38th international conference on software engineering*, pages 1157–1168, 2016.
- [22] Hinrich Schütze, Christopher D Manning, and Prabhakar Raghavan. *Introduction to information retrieval*, volume 39. Cambridge University Press Cambridge, 2008.
- [23] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units, 2016.
- [24] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [25] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859*, 2021.
- [26] Peipei Xia, Li Zhang, and Fanzhang Li. Learning similarity with cosine similarity ensemble. *Information sciences*, 307:39–52, 2015.
- [27] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223*, 2023.
- [28] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*, 2019.